

vcloset — Virtual Try-On 서비스

IT 개발 전(全) 과정 백서

기획 · 설계 · 구현 · 배포 · 테스트(알파/엔진) · 출시

이재환 · 2026. 06. 12 · github.com/JHWwal

목차

목차.....	2
1. 프로젝트 개요.....	3
2. 기획 (Planning).....	4
2.1 문제 정의와 배경.....	4
2.2 타겟 사용자와 핵심 가치.....	4
2.3 기능 요구사항.....	4
2.4 제약 조건 — 무료 개발 원칙.....	4
2.5 로드맵.....	4
3. 설계 (Design).....	6
3.1 시스템 아키텍처.....	6
3.2 추론 백엔드 추상화 — 4단 전환.....	6
3.3 데이터 모델 (Prisma 5종).....	6
3.4 크레딧 · 트랜잭션 설계.....	6
3.5 캐시 설계.....	7
4. 구현 (Implementation).....	8
4.1 주차별 구현 내역.....	8
4.2 주요 기술 결정과 이유.....	8
4.3 HF Space 브릿지 (06.12 추가).....	8
5. 배포 (Deployment).....	10
5.1 실행 환경 구성.....	10
5.2 크로스플랫폼 이식 (macOS → Windows, 06.12).....	10
5.3 추론 인프라 옵션 비교.....	10
5.4 운영 전환 계획.....	10
6. 테스트 (Testing).....	11
6.1 알파 테스트 — 사용자 흐름 E2E (06.12).....	11
6.2 엔진 테스트 — 합성 품질·정합성 (06.12).....	11
6.3 테스트 결과 요약.....	12
7. 출시 (Release).....	13
7.1 현재 상태.....	13
7.2 출시 전 체크리스트.....	13
7.3 출시 전략.....	13
8. 회고 및 교훈.....	13

1. 프로젝트 개요

vcloset은 사용자가 자신의 전신(또는 반신) 사진과 옷 사진을 업로드하면, AI가 그 옷을 실제로 입은 모습을 합성해 보여주는 개인 가상 옷장 서비스다. 2020년에 수행했던 가상착장 멀티모델 파이프라인(YOLO + OpenPose + VITON)을 2026년의 디퓨전 기반 VTON(CatVTON / IDM-VTON) 위에서 재구축한 프로젝트로, 단순 데모가 아니라 회원·크레딧·결제 구조를 갖춘 서비스 형태를 목표로 한다.

항목	내용
서비스 한 줄 정의	사진 한 장 + 옷 한 장 → AI 합성으로 입은 모습 미리보기
개발 인원 / 형태	1인 풀스택 (기획·설계·구현·테스트 전담)
프론트/백엔드	Next.js 16 (App Router) · TypeScript · Tailwind v4 · NextAuth v5
데이터	Prisma ORM · SQLite(개발) → PostgreSQL(운영 전환 예정)
AI 추론	IDM-VTON / CatVTON (사전학습 디퓨전 VTON) · rembg 배경 제거
추론 인프라	Replicate · Modal · Colab(T4) · HF Space — 4종 백엔드 전환 가능
개발 원칙	무료 우선 — 로컬 개발 + 무료 GPU(Colab/HF), 운영 전환 시 유료 전환

역량 구분(정직한 표기): 디퓨전 합성 모델 자체는 사전학습 오픈소스를 활용했다. 본 프로젝트에서 직접 설계·구현한 것은 그 모델을 서비스로 만드는 전체 시스템 — 인증/옷장/크레딧 원장/캐시/실패 환불/멀티 백엔드 폴백/전처리 파이프라인 — 이다.

2. 기획 (Planning)

2.1 문제 정의와 배경

- 온라인 의류 구매 시 “내가 입으면 어떨까”를 확인할 방법이 없다 — 반품률과 구매 망설임의 핵심 원인.
- 기존 VTON 데모는 연구용 노트북/허깅페이스 데모 수준 — 일반 사용자가 회원가입하고 옷장을 관리하며 쓰는 ‘서비스’ 형태가 없다.
- 2020 년의 가상착장 파이프라인 경험(YOLO + OpenPose + VITON, Affine Warping)을 디퓨전 세대 기술로 발전시켜 포트폴리오 서사를 완성한다.

2.2 타겟 사용자와 핵심 가치

- 타겟: 온라인 쇼핑 비중이 높은 일반 사용자 (모바일/웹).
- 핵심 가치: ① 내 사진 기반의 사실적 합성 ② 옷장 단위의 이력 관리(룩북) ③ 크레딧로 비용 통제.

2.3 기능 요구사항

기능	요구사항
인증	이메일/비밀번호 가입·로그인 (NextAuth v5 Credentials), 가입 시 무료 크레딧 5장
온보딩	프로필(인물) 사진 업로드, 대표 사진 지정
옷장	옷 등록(카테고리 6종)·갤러리·삭제, 업로드 시 배경 자동 제거
가상 피팅	인물 × 옷 선택 → 합성 실행, 1회 1크레딧, 동일 조합 캐시
룩북	합성 이력 모음, 결과 영구 보존
크레딧	원장(ledger) 기반 차감/환불, 잔액 부족 시 402

2.4 제약 조건 — 무료 개발 원칙

- 개발 머신(GPU 없음/저사양)에서 UI·백엔드·DB 전부 개발 가능해야 한다 → 추론은 외부로 분리.
- 무료 GPU(Colab T4, HF Space)로 합성 검증, 운영 단계에서만 유료(Replicate) 전환.
- 스토리지는 로컬(public/uploads) → 운영 전환 시 R2/S3.

2.5 로드맵

주차	목표	상태
Week 1	프로젝트 골격, 인증, 옷장 CRUD, 업로드, 피팅 placeholder	완료
Week 2	rembg 배경 제거(Node), 룩북 페이지, try-on 캐싱	완료
Week 3	Colab CatVTON + FastAPI + ngrok 연결, 결과 저장, 실패 시 환불	완료
(추가)	Windows 이식 + HF Space 브릿지 + 알파/엔진 테스트	완료 (06.12)
Week 4	Stripe 결제, Rate Limit, 워터마크	예정
Week 5	공유, 모바일 최적화, 베타 오픈	예정

3. 설계 (Design)

3.1 시스템 아키텍처

Next.js 16 App Router 모놀리스(웹 UI + API Route)와 외부 추론 서버를 분리한 2계층 구조다. 무거운 디퓨전 추론을 앱 프로세스 밖으로 분리해, GPU 없는 개발 환경에서도 전체 서비스가 동작하고, 추론 백엔드를 환경변수만으로 교체할 수 있다.

[Browser] → [Next.js 16 — UI · API Routes · NextAuth · Prisma] → (HTTP/JSON) → [추론 서버 — Replicate API | Modal | Colab FastAPI | HF Space 브릿지] → [디퓨전 VTON 모델]

3.2 추론 백엔드 추상화 — 4 단 전환

try-on API는 환경변수 상태로 추론 모드를 자동 선택한다(pickMode). 이 추상화 덕에 개발(placeholder) → 검증(무료 GPU) → 운영(유료 API)으로 코드 수정 없이 전환된다.

모드	조건	용도
replicate	REPLICATE_API_TOKEN 존재	운영 (안정, 유료)
modal	INFERENCE_URL + INFERENCE_MODE=modal	검증 (서버리스 GPU)
colab	INFERENCE_URL 존재	검증 (무료 T4 / HF Space 브릿지)
placeholder	아무것도 없음	개발 (옷 이미지 그대로 반환)

3.3 데이터 모델 (Prisma 5 종)

모델	역할
User	계정 · plan · creditBalance(잔액 캐시)
ProfileImage	인물 사진, 대표(isPrimary) 지정
Garment	옷 — url(배경 제거본) · originalUrl(원본) · category · name
TryOnSession	합성 1회 — 상태(PROCESSING/DONE/FAILED) · resultUrl · durationMs · cacheHit · errorMessage
CreditLedger	크레딧 원장 — amount(±) · reason(TRY_ON/REFUND/ADMIN) · balanceAfter · 참조(refType/refId)

3.4 크레딧 · 트랜잭션 설계

- 차감과 세션 생성은 `prisma.$transaction` 으로 원자화 — 부분 실패로 인한 정합성 깨짐 방지.
- 추론 실패 시 같은 트랜잭션 패턴으로 환불(+1) + REFUND 원장 기록 + 세션 FAILED 마킹.
- 원장에 `balanceAfter` 를 함께 기록해 잔액 추적·감사가 가능하도록 설계.

3.5 캐시 설계

- 동일 (인물 사진 × 옷) 조합의 DONE 세션이 있으면 추론 없이 기존 결과를 재사용(cacheHit) — GPU 비용과 대기시간 절감.

4. 구현 (Implementation)

4.1 주차별 구현 내역

구분	구현 내용
Week 1	App Router 라우트 6종(랜딩/가입/로그인/온보딩/옷장/피팅) · NextAuth Credentials · 옷장 CRUD API · 파일 업로드 · 피팅 placeholder 동작
Week 2	@imgly/background-removal-node로 옷 업로드 시 배경 자동 제거 · 룩북 페이지 · (인물×옷) 캐시
Week 3	Colab CatVTON FastAPI 추론 서버(ngrok) 연동 · base64 왕복 · 결과 PNG 저장 · 실패 시 자동 환불
06.12	Windows 이식 · HF Space 브릿지 서버 · 피팅 결과 세션 복원(상태 버그 수정) · 데모 시드 정합성 수정
06.12	CatVTON 전환으로 하의(BOTTOM) 지원 · 카테고리 UI를 상의·하의로 정리 · 여름 테마 리디자인 · projects/vcloset 으로 코드·문서 정리

4.2 주요 기술 결정과 이유

결정	이유
추론을 HTTP 규약으로 분리	GPU 없는 머신에서 전체 개발 가능 · 백엔드 4종 교체 자유
크레딧을 원장(ledger)으로	단순 카운터와 달리 차감/환불/지급 이력이 감사 가능
옷 업로드 시 rembg 전처리	VTON 입력 품질 향상(배경 노이즈 제거) · 원본은 별도 보존
cloth_type 매핑 테이블	백엔드마다 다른 카테고리 체계(Replicate/CatVTON)를 한 곳에서 변환
결과를 파일+세션으로 영구화	룩북·캐시·환불 분쟁 대응의 근거 데이터

4.3 HF Space 브릿지 (06.12 추가)

API 토큰 없이 실험성을 검증하기 위해, vcloset의 추론 규약(POST /try-on, JSON in/out)을 Hugging Face 공개 VTON Space(gradio)로 중계하는 FastAPI 브릿지를 구현했다 (inference/hf_space_bridge.py). 기존 앱 코드는 한 줄도 수정하지 않고 .env의 INFERENCE_URL 만 지정해 연결했다 — 3.2의 추상화 설계가 실전에서 검증된 사례다.

초기에는 상의 전용 IDM-VTON Space를 사용했으나, 하의 지원을 위해 cloth_type(upper/lower/overall)을 받는 CatVTON 공식 Space로 전환했다. 전환 과정에서 ① 옷 이미지의 RGBA→JPEG 재저장 충돌(모든 입력을 RGB로 정규화) ② Space가 요구하는 마스크 레

이어(투명 레이어 전송 → 자동 마스킹 유도) 두 가지를 해결했다. 앱 측은 BOTTOM→lower 카테고리 매핑만으로 하의 합성이 동작한다.

5. 배포 (Deployment)

5.1 실행 환경 구성

- 환경변수(.env): DATABASE_URL · AUTH_SECRET(랜덤 32 바이트) · AUTH_URL · INFERENCE_URL.
- DB 초기화: prisma migrate deploy → 데모 시드(seed-demo.ts)로 인물 4 장 + 옷 5 벌(상의 4·하의 1) + 데모 계정 자동 구성.
- 기동: next dev (개발) — 운영 빌드는 next build + next start.

5.2 크로스플랫폼 이식 (macOS → Windows, 06.12)

개발은 macOS에서 진행했고, 06.12에 Windows 10 환경으로 이식해 전 기능을 재검증했다. 이식 과정에서 발견·해결한 항목:

이슈	해결
Node.js 부재	winget으로 Node LTS(v24) 설치
npm script의 유닉스식 env 주입 (NODE_OPTIONS=...)	Windows에서는 동작하지 않음 → 기동 배치(run_dev.bat)에서 환경변수 설정 후 실행
rembg ONNX 모델 첫 다운로드	Windows에서도 정상 동작 확인 (시드 시 4벌 모두 배경 제거 성공)

5.3 추론 인프라 옵션 비교

옵션	비용	속도(실측/예상)	용도
HF Space 브릿지	무료	26~49초	데모 · 검증
Colab T4 + ngrok	무료	30~60초	검증 (세팅 10~15분)
Replicate API	회당 약 \$0.05	20~40초	운영 · 시연 안정성
placeholder	무료	즉시	UI 개발

5.4 운영 전환 계획

- DB: SQLite → PostgreSQL (스키마는 Prisma 로 동일 유지).
- 스토리지: public/uploads → Cloudflare R2/S3 (storage.ts 추상화 계층 이미 존재).
- 호스팅: Vercel(웹) + Replicate(추론) 조합이 1 인 운영에 최적.

6. 테스트 (Testing)

6.1 알파 테스트 — 사용자 흐름 E2E (06.12)

실제 사용자 시나리오를 처음부터 끝까지 수행하며 기능·UX를 검증했다: 로그인 → 옷장(필터·삭제) → 피팅(옷 미리보기 모달 → 선택 → 합성) → 크레딧 차감 → 룩북 이력 확인.

번호	발견 결함	조치
A-1	합성 완료 후 페이지가 리로드되면 결과 패널이 초기 상태로 돌아감 (결과가 메모리 상태에만 존재)	마운트 시 서버의 마지막 완료 세션을 조회해 결과·선택 상태를 복원하도록 수정 → 재검증 통과
A-2	데모 시드 옷 이름과 실제 이미지 불일치 (예: 'Striped shirt'가 실제로는 검은 로고 티셔츠)	시드 데이터 이름 전면 정정, 옷 단독 사진이 아닌 항목 제외

6.2 엔진 테스트 — 합성 품질·정합성 (06.12)

상의·하의 정상 경로를 실험성으로 검증해 '선택한 옷 = 입혀진 옷'을 이미지 단위로 확인했다. 그래픽 프린트(고양이 일러스트)·로고 위치 보존, 하의 합성 시 상의 유지까지 확인.

조합	소요 시간	상태	비고
남성 × 하늘색 티 (상의)	49.0초	DONE	첫 호출(콜드)
남성 × 검은 705 로고 티 (상의)	32.1초	DONE	로고 위치·형태 보존
여성 × 고양이 그래픽 티 (상의)	26.8초	DONE	프린트 디테일 보존
전신 모델 × 네이비 진 (하의)	20.6초	DONE	상의 유지, 하의만 교체

추가로 케이스 단위 자동 스위트(inference/engine_test_suite.py)로 입력 검증·강건성·정합성을 점검했다. 모든 케이스는 재실행 가능하다.

그룹	케이스	결과
입력 검증 (V)	손상된 base64 · 비이미지 데이터 · 빈 페이로드	3/3 — GPU 도달 전 차단
엔진 코어 (E)	상의·하의 기준선 · 투명 RGBA 옷(rembg 산출물)	통과 — RGB 정규화 후 정상 합성
정합성 (I)	크레딧 장부 불변식 $\Sigma(\text{amount})=\text{잔액}$	불일치 1건 검출 →

		원인 규명 후 정정
--	--	------------

E-1 환각 결함 분석(핵심 디버깅): 초기 테스트에서 검은 로고 티를 선택했는데 결과에 존재하지 않는 줄무늬가 생성됐다. 원인은 VTON 모델이 옷 이미지와 함께 받는 텍스트 설명 (garment_desc)에 잘못된 옷 이름('Striped shirt')이 전달되어, 모순된 텍스트 조건이 이미지에 없는 디테일을 생성한 것. 조치: 브릿지에서 텍스트 설명을 카테고리 기반 중립 표현으로 강제해 이미지가 합성을 지배하도록 수정 → 동일 조합 재합성으로 일치 확인.

실패 경로(환불) 테스트: 추론 서버를 의도적으로 내린 상태에서 합성을 실행 → HTTP 502 응답, 크레딧 자동 환불(원장: TRY_ON -1 → REFUND +1, balanceAfter 일치), 세션 FAILED + 오류 메시지 기록을 모두 확인했다.

6.3 테스트 결과 요약

항목	결과
E2E 사용자 흐름	통과 (가입~룩북 전 구간)
실합성 정합성 (옷 3종 × 인물 2명)	통과 — 선택 옷과 결과 일치, 프린트 보존
합성 소요 시간 (무료 HF GPU)	26~49초, 캐시 적중 시 즉시
실패 시 복구	통과 — 자동 환불 + FAILED 세션 기록
발견 결함	3건 (A-1, A-2, E-1) — 전건 수정·재검증 완료

7. 출시 (Release)

7.1 현재 상태

Week 3 + 엔진 검증까지 완료된 베타 직전 단계. 핵심 가치(실합성)와 과금 기반(크레딧 원장)이 동작하며, 남은 것은 결제·보호장치·배포 인프라다.

7.2 출시 전 체크리스트

영역	항목
결제	Stripe(또는 Toss) 크레딧 충전 — 원장 구조는 이미 대응됨
보호	Rate Limit(추론 남용 방지) · 결과 워터마크 · 업로드 이미지 검증(인물 여부)
인프라	PostgreSQL 전환 · R2/S3 스토리지 · Vercel 배포 · Replicate 키
품질	모바일(iOS Safari) 최적화 · 공유 기능 · 합성 대기 UX(진행률)
법무	업로드 이미지 보존·삭제 정책 · 초상권 고지

7.3 출시 전략

- 1 단계 — 클로즈드 베타: 무료 크레딧 5 장, HF/Colab 추론으로 비용 0 원 운영, 피드백 수집.
- 2 단계 — 오픈 베타: Replicate 전환 + 크레딧 충전 오픈, 합성 단가 대비 크레딧 가격 책정.
- 3 단계 — 정식: 카테고리 확장(하의·원피스), 모바일 최적화, 공유 바이럴 루프.

8. 회고 및 교훈

- **활용과 구현을 구분해 말하기** — 합성 모델은 사전학습 오픈소스를 '활용'했고, 직접 '구현'한 것은 서비스 시스템 전체다. 이 구분을 명확히 하는 것이 기술 신뢰의 출발점이다.
- **모순된 입력은 환각을 만든다** — 멀티모달 모델에서 텍스트 조건과 이미지 조건이 충돌하면 모델은 그럴듯한 거짓을 생성한다(E-1). 입력 정합성 검증은 모델 성능만큼 중요하다.
- **추상화는 공짜 보험이다** — 추론 백엔드를 HTTP 규약으로 추상화해 둔 덕에, 6 개월 뒤 전혀 다른 인프라(HF Space)를 코드 수정 없이 연결했다.
- **실패 경로가 곧 신뢰다** — 환불·FAILED 기록·캐시 같은 '안 보이는 코드'가 데모와 서비스를 가르는 차이였다.

- **다음 고도화 후보** — CLIP 제로샷 옷 카테고리 자동 분류, 업로드 사진 인물 검증(YOLO), 합성 품질 자동 평가.